47-team-git-workflow

Figure 1: Team Git Workflow Banner

Modul 47: Team Git Workflow

> Kelola kode tim profesional — branching strategy, Pull Request, code review, conflict resolution, CI/CD otomatis.

Informasi Modul

Item	Detail
Tingkat	□ Intermediate
Durasi	16 Jam (4 sesi × 4 jam)

Item	Detail
Prasyarat	Lulus Modul 5 (Git & GitHub Dasar)
Tools	Git, GitHub, VS Code, GitHub Actions, GitHub Projects
Output	Pull Request workflow real team + CI/CD pipeline

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, peserta mampu:

1. Memilih dan menerapkan branching strategy (GitFlow, GitHub Flow, Trunk-based)
2. Menulis commit dengan Conventional Commits dan mengelola semantic versioning
3. Membuat dan melakukan code review Pull Request dengan checklist profesional
4. Menyelesaikan merge conflict dan melakukan rebase workflow
5. Setup GitHub Actions CI/CD pipeline dan GitHub Projects board

Materi Pembelajaran

Sesi	Topik	File
1	**Branching Strategy** — GitFlow vs GitHub Flow vs Trunk-based, branch naming, Conventional Commits, semantic versioning, tagging & release	01-branching-strategy.md
2	**Pull Request Workflow** — PR lifecycle, description template, code review checklist, merge strategies, protected branches, CODEOWNERS	02-pr-workflow.md
3	**Conflict Resolution** — merge conflict types, resolve di VS Code, rebase workflow, cherry-pick, revert, reset	03-conflict-resolution.md

Sesi	Topik	File
4	**GitHub Actions & Project Management** — CI/CD pipeline, workflow YAML, matrix build, GitHub Projects, issue/PR templates	04-github-actions.md

Output Akhir

Setelah menyelesaikan seluruh sesi, peserta menghasilkan:

- **Branching Strategy Dokumen** — strategi branch tim + branch naming convention
- **Pull Request Real Team** — PR dengan deskripsi, review, diskusi, merge
- **Conflict Resolution Practice** — PR dengan conflict yang berhasil di-resolve
- **CI/CD Pipeline** — GitHub Actions workflow untuk lint, test, build, deploy
- **GitHub Project Board** — Issues, milestones, labels, automation

AI Prompt Exercises

Sesi	Prompt AI
1	“Explain the difference between GitFlow, GitHub Flow, and Trunk-based development. When would you choose each one for a team of 5 developers vs 20 developers?”
2	“Generate a Pull Request description template for a team project — include sections for what, why, how, testing, and screenshots in Indonesian.”
3	“Explain merge conflict types (content, structural, rename/delete) and how to resolve each one using VS Code merge editor. Give examples.”

Sesi	Prompt AI
4	“Generate a GitHub Actions CI workflow for a Node.js project with linting, type checking, testing, and build stages using matrix strategy for multiple Node versions.”

Sesi 1: Branching Strategy

> Pilih strategi branch yang tepat, beri nama branch konsisten, tulis commit terstruktur, dan kelola versi rilis profesional.

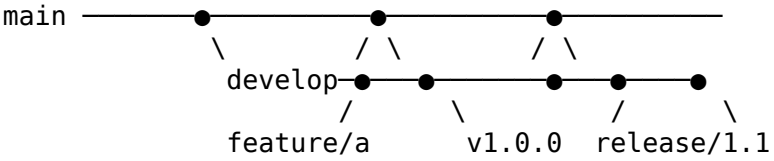
Durasi: 4 jam | **Output**: Branching strategy dokumen + repo dengan conventional commits

---

1.1 Branching Strategy Overview

Tiga strategi branching utama yang dipakai tim engineering:

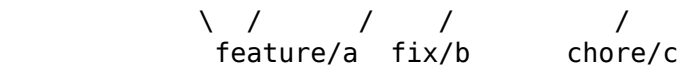
GitFlow



- **main** — kode produksi, hanya menerima merge dari `release` atau `hotfix`
- **develop** — integration branch untuk fitur yang belum dirilis
- **feature/** — branch untuk fitur baru, branch dari `develop`, merge balik ke `develop`
- **release/** — persiapan rilis, branch dari `develop`, merge ke `main` dan `develop`
- **hotfix/** — perbaikan urgent di produksi, branch dari `main`, merge ke `main` dan `develop`

Cocok untuk: Proyek dengan rilis terjadwal, tim besar, produk stabil.

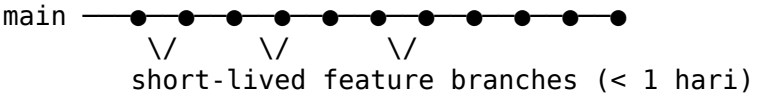
GitHub Flow



- Hanya satu branch abadi: `main`
- Semua pekerjaan di branch fitur → Pull Request → review → merge ke `main`
- Deploy langsung dari `main` atau dari branch fitur

Cocok untuk: Tim kecil, deployment kontinu (CI/CD), startup.

Trunk-based Development



- Developer commit langsung ke `main` atau branch pendek (short-lived)
- Branch fitur maksimal 1-2 hari sebelum merge
- Feature flags untuk menyembunyikan kode yang belum siap

Cocok untuk: CI/CD intensif, tim sangat agile, deployment multiple kali sehari.

Perbandingan

Aspek	GitFlow	GitHub Flow	Trunk-based
Jumlah branch abadi	2 (main, develop)	1 (main)	1 (main)
Umur feature branch	Hari-minggu	Hari	Jam-1 hari
Kompleksitas	Tinggi	Rendah	Rendah
Cocok untuk	Rilis terjadwal	Deploy kontinu	Deploy sangat sering
Release manage-ment	Release branch	Tag / branch dari main	Feature flags

---

1.2 Branch Naming Convention

Format konsisten biar branch mudah dilacak:

`<type>/<deskripsi-singkat>`

Gunakan **kebab-case** untuk deskripsi.

Tipe Branch

Tipe	Prefix	Contoh
Feature	`feature/`	`feature/login-page`, `feature/api-user-crud`
Bug Fix	`fix/`	`fix/navbar-overflow`, `fix/null-pointer-auth`
Hotfix	`hotfix/`	`hotfix/payment-gateway-down`, `hotfix/security-patch`
Chore	`chore/`	`chore/update-deps`, `chore/refactor-db-config`
Docs	`docs/`	`docs/api-readme`, `docs/contributing-guide`
Release	`release/`	`release/v1.2.0`, `release/v2.0.0-rc.1`

Contoh Buruk vs Baik

Buruk	Baik
`my-branch`	`feature/login-page`
`fix-bug`	`fix/navbar-overflow-mobile`
`wip`	`chore/update-tailwind-v3`
`tes`	`docs/setup-guide`

1.3 Conventional Commits

Format standar pesan commit yang terstruktur:

```
<type>(<optional scope>): <description>

[optional body]
```

```
[optional footer(s)]
```

Tipe Commit

Tipe	Kapan Pakai
`feat:`	Fitur baru
`fix:`	Perbaikan bug
`chore:`	Tugas rutin (update deps, config)
`docs:`	Dokumentasi
`refactor:`	Refaktor kode tanpa perubahan behavior
`style:`	Formatting, whitespace (bukan CSS)
`test:`	Nambah / update test
`perf:`	Optimasi performa
`ci:`	Config CI/CD
`build:`	Build system / dependencies

Contoh

```
feat(auth): add login page with JWT validation

Implementasi form login dengan validasi JWT token.
Email dan password divalidasi client-side sebelum dikirim.

Closes #42
```

```
fix(navbar): fix overflow on mobile viewport

Navbar item keluar layar di viewport < 375px.
Ganti flex-wrap dan tambah min-width agar responsif.

Fixes #87
```

```
chore(deps): upgrade axios from 0.27 to 1.6

Perlu update karena security vulnerability CVE-2023-XXXXX.
```

```
docs: add API endpoint documentation for /users

Lengkapi README dengan daftar endpoint, contoh request/response.
```

BREAKING CHANGE

Gunakan `BREAKING CHANGE` di footer atau tambahkan `!` setelah type/scope:

```
feat(api)!: change user response format

BREAKING CHANGE: Field 'username' renamed to 'handle'.
Semua client perlu update mapping.

refactor!(db): migrate from MySQL to PostgreSQL

BREAKING CHANGE: Semua query SQL perlu diadaptasi ke dialect PostgreSQL.
```

1.4 Semantic Versioning (SemVer)

Format: `MAJOR.MINOR.PATCH` — `v1.4.2`

Komponen	Increment Ketika
MAJOR	Breaking change — API tidak backward compatible
MINOR	Fitur baru — backward compatible
PATCH	Bug fix — backward compatible

Pre-release Tags

```
v1.0.0-alpha.1
v1.0.0-beta.2
v1.0.0-rc.3
```

- alpha — pengembangan awal, fitur belum lengkap
- beta — fitur lengkap, masih ada bug
- rc (release candidate) — stabil, siap testing final

Contoh Versioning

```
v1.0.0        ← Rilis pertama
v1.1.0        ← Tambah fitur baru (minor)
v1.1.1        ← Bug fix
v2.0.0-alpha  ← Breaking change (masih alpha)
v2.0.0-rc.1   ← Siap testing
v2.0.0        ← Rilis major
```

1.5 Tagging & Release Branch

Membuat Tag

```
# Lightweight tag
git tag v1.0.0
```

```
# Annotated tag (recommended — simpan metadata)
git tag -a v1.0.0 -m "Release v1.0.0 - First stable version"

# Push tag ke remote
git push origin v1.0.0

# Push semua tag
git push --tags
```

Release Branch Workflow (GitFlow)

```
# 1. Branch release dari develop
git checkout develop
git checkout -b release/v1.0.0

# 2. Final bug fix, update version
npx standard-version --release-as 1.0.0

# 3. Merge ke main + tag
git checkout main
git merge --no-ff release/v1.0.0
git tag -a v1.0.0 -m "Release v1.0.0"

# 4. Merge balik ke develop
git checkout develop
git merge --no-ff release/v1.0.0

# 5. Hapus release branch
git branch -d release/v1.0.0
```

GitHub Release

Selain tag lokal, buat **GitHub Release** di UI:

1. Buka repo → **Releases** → **Create a new release**
2. Pilih tag, isi title + release notes
3. Upload binary/lampiran jika perlu
4. Publish

1.6 Diagram: Branching Strategy

GitFlow Mermaid

```
gitGraph
   commit
   branch develop
   commit
   branch feature/login
   commit
   commit
   checkout develop
   merge feature/login
   branch release/v1.0.0
   commit
   checkout main
   merge release/v1.0.0
   tag "v1.0.0"
   checkout develop
   merge release/v1.0.0
   branch hotfix/security
   commit
   checkout main
   merge hotfix/security
   tag "v1.0.1"
   checkout develop
   merge hotfix/security
```

GitHub Flow Mermaid

```
gitGraph
   commit
   commit
   branch feature/login
   commit
   commit
   checkout main
   merge feature/login
   commit
   tag "v1.0.0"
   branch fix/navbar
   commit
   checkout main
   merge fix/navbar
   commit
   tag "v1.0.1"
```

1.7 Latihan

Latihan 1: Setup Branching Strategy

1. Buat repo baru di GitHub (public atau private)
2. Inisialisasi dengan README
3. Clone ke lokal

```
git clone https://github.com/username/latihan-git-workflow.git
cd latihan-git-workflow
```

4. Setup **GitFlow**:

```
# Install git-flow (Ubuntu/Debian)
sudo apt install git-flow

# Init git-flow di repo
git flow init
# Accept default values (Enter all)

# Buat feature branch dengan git-flow
git flow feature start landing-page
```

5. Alternatif setup **GitHub Flow** manual:

```
# Branch fitur langsung dari main
git checkout main
git pull origin main
git checkout -b feature/landing-page
```

Latihan 2: Practice Conventional Commits

1. Di branch `feature/landing-page`, buat file `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Landing Page</title>
</head>
<body>
  <h1>Selamat Datang</h1>
  <p>Landing page kami.</p>
</body>
</html>
```

2. Stage dan commit:

```
git add index.html
git commit -m "feat(landing): add landing page with welcome section"
```

3. Edit file, tambah CSS:

```
<style>
  body { font-family: sans-serif; text-align: center; padding: 2rem; }
  h1 { color: #2563eb; }
</style>
```

4. Commit kedua:

```
git add index.html
git commit -m "style(landing): add basic styling with blue heading"
```

5. Buat beberapa branch dan commit dengan tipe berbeda:

```
# Branch fix
git checkout main
git checkout -b fix/typo-welcome
# Edit perbaiki typo
git add .
git commit -m "fix(landing): fix typo in welcome message"

# Branch chore
git checkout main
git checkout -b chore/add-gitignore
echo "node_modules/" > .gitignore
git add .gitignore
git commit -m "chore: add node_modules to gitignore"
```

Latihan 3: Tagging & Verifikasi

```
# Merge branch ke main
git checkout main
git merge --no-ff feature/landing-page
git merge --no-ff fix/typo-welcome

# Buat tag
git tag -a v0.1.0 -m "v0.1.0 - Landing page dasar"

# Push semua
git push origin main --tags

# Lihat log
git log --oneline --graph --all
```

```
# Lihat tag
git tag -l -n
```

Checklist Output Sesi 1

- ☐ Repo dengan strategi branch (GitFlow/GitHub Flow) sudah diinisialisasi
- ☐ Minimal 3 branch berbeda dengan naming convention benar
- ☐ Minimal 5 commit dengan format Conventional Commits
- ☐ Minimal 2 tipe commit berbeda (feat, fix, chore, dll)
- ☐ Tag semantic versioning sudah dibuat
- ☐ Branch strategy didokumentasikan di README repo

Referensi

- GitFlow - Vincent Driessen
- GitHub Flow Guide
- Trunk-based Development
- Conventional Commits
- Semantic Versioning
- GitFlow CLI Cheatsheet

Sesi 2: Pull Request Workflow

> Kuasai alur Pull Request profesional — dari deskripsi hingga merge, dengan code review yang efektif.

Durasi: 4 jam | **Output**: Pull Request real dengan review, diskusi, dan merge

---

2.1 PR Lifecycle

Alur standar Pull Request di tim engineering:

```
Create PR → Assign Reviewer → Review → Request Changes / Approve → Merge → Deploy
                                ↓
                          Discuss & Iterate
```

Tahapan Detail

1. **Create** — Push branch → buka GitHub → klik “Compare & Pull Request”

2. **Describe** — Isi template PR (what, why, how, testing)
3. **Assign** — Assign reviewer (otomatis via CODEOWNERS atau manual)
4. **Review** — Reviewer baca kode, tinggalkan komentar
5. **Discuss** — Author jawab komentar, push perubahan
6. **Approve** — Reviewer setuju, klik Approve
7. **Merge** — Author merge setelah semua checklist terpenuhi
8. **Deploy** — Otomatis via CI/CD atau manual

PR Size Matters

Ukuran PR	Guidelines
Small (< 200 lines)	Ideal — mudah review, cepat merge
⚠ **Medium (200-500 lines)**	Masih OK — usahakan dipecah
Large (> 500 lines)	Red flag — minta author split

> **Aturan praktik**: Satu PR = satu fitur/perbaikan. Jangan campur refactor + fitur baru + fix di satu PR.

2.2 PR Description Template

Template standar yang dipakai tim profesional. Simpan file di `.github/pull_request_template.md`:

```
## Deskripsi

<!-- Apa yang diubah di PR ini? Jelaskan secara singkat. -->

## Related Issue

<!-- Closes #issue_number, Fixes #issue_number, atau Related to #issue_number --

Closes #

## Perubahan

- [ ] Fitur baru
- [ ] Perbaikan bug
- [ ] Refactor
- [ ] Dokumentasi
- [ ] Dependencies

## Type of Change
```

```
- [ ] feat: — fitur baru (non-breaking)
- [ ] fix: — perbaikan bug (non-breaking)
- [ ] chore: — tugas rutin
- [ ] docs: — dokumentasi
- [ ] refactor: — refaktor
- [ ] BREAKING CHANGE — perubahan tidak backward compatible

## How to Test

<!-- Langkah-langkah untuk testing PR ini -->

1. Checkout branch: `git checkout <branch>`
2. Install dependencies: `npm install`
3. Jalankan: `npm run dev`
4. Buka http://localhost:3000
5. Verifikasi: ...

## Screenshots (jika ada)

| Sebelum | Sesudah |
|---------|---------|
|         |         |

## Checklist

- [ ] Kode sudah di-test lokal
- [ ] Tidak ada warning/error baru
- [ ] Unit test ditambahkan (jika perlu)
- [ ] Dokumentasi diupdate (jika perlu)
- [ ] Branch sudah up-to-date dengan main
```

Cara Setup

```
mkdir -p .github
touch .github/pull_request_template.md
# Paste template di atas
git add .github/pull_request_template.md
git commit -m "docs: add PR template"
```

2.3 Code Review Checklist

General

- ☐ Apakah kode sesuai dengan requirement/ticket?
- ☐ Apakah ada duplicate code yang bisa di-extract?
- ☐ Apakah penamaan variable/fungsi jelas dan konsisten?
- ☐ Apakah ada dead code / komentar yang tidak perlu?
- ☐ Apakah ada perubahan tidak terduga di file lain?

Logic & Correctness

- ☐ Apakah logic sudah benar untuk semua edge case?
- ☐ Apakah ada potential null pointer / undefined access?
- ☐ Apakah error handling sudah memadai?
- ☐ Apakah tidak ada race condition (async code)?
- ☐ Apakah validasi input sudah dilakukan?

Style & Consistency

- ☐ Apakah mengikuti coding style tim (ESLint/Prettier)?
- ☐ Apakah format import konsisten?
- ☐ Apakah tidak ada hardcoded values (magic numbers/strings)?
- ☐ Apakah CSS/Tailwind mengikuti convention tim?

Performance

- ☐ Apakah tidak ada infinite loop / recursion tanpa batas?
- ☐ Apakah query database sudah optimal (N+1 problem)?
- ☐ Apakah tidak ada render berulang (React useEffect missing deps)?
- ☐ Apakah assets/bundles terlalu besar?

Security

- ☐ Apakah user input sudah di-sanitize (XSS, SQL injection)?
- ☐ Apakah API endpoint punya autentikasi/otorisasi?
- ☐ Apakah secret/token tidak hardcoded?
- ☐ Apakah ada rate limiting untuk public endpoint?

Test Coverage

- ☐ Apakah ada unit test untuk logic baru?
- ☐ Apakah test passing di CI?
- ☐ Apakah snapshot test diupdate (jika pakai)?
- ☐ Apakah ada integration test untuk endpoint baru?

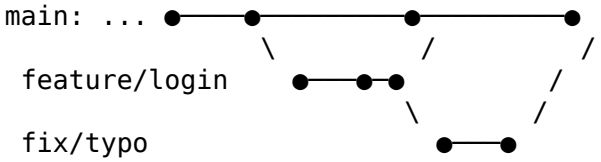
Writing Review Comments

Tipe Komentar	Cara Menulis
Suggestion	“Sebaiknya pake `map()` instead of `forEach()` biar immutable”
Question	“Kenapa pake `var` instead of `const`? Ada alasan spesifik?”
Issue	“Ini potential XSS — input user langsung di-render tanpa sanitize. Pake DOMPurify.”
Praise	“Bagus! Error handling-nya lengkap.”

2.4 Merge Strategies

Merge Commit

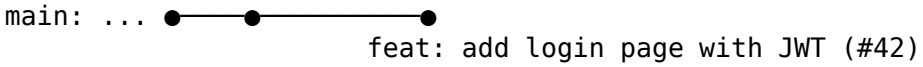
```
git checkout main
git merge --no-ff feature/login
```



- □ Mempertahankan histori branch lengkap
- □ Setiap merge punya commit terpisah
- □ Graf bisa kompleks untuk tim besar
- Cocok untuk: GitFlow, histori lengkap

Squash & Merge

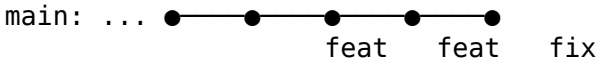
```
# Semua commit fitur jadi satu commit
git merge --squash feature/login
git commit -m "feat: add login page with JWT (#42)"
```



- □ Histori bersih dan linear
- □ Commit fitur jadi satu kesatuan logis
- □ Kehilangan detail commit individual
- Cocok untuk: GitHub Flow, tim kecil

Rebase & Merge

```
git checkout feature/login
git rebase main
git checkout main
git merge feature/login
# atau fast-forward saja
```



- ✅ Histori linear sempurna
- ✅ Setiap commit tetap ada
- ❌ Perlu force push (rebase)
- Cocok untuk: Trunk-based, tim disiplin

Perbandingan

Aspek	Merge Commit	Squash & Merge	Rebase & Merge
Histori branch	✅ Lengkap	❌ Hilang	✅ Lengkap
Histori linear	❌ Graf bercabang	✅ Linear	✅ Linear
Commit granular	✅ Semua visible	❌ Satu commit	✅ Semua visible
Force push	✅ Tidak perlu	✅ Tidak perlu	❌ Perlu
Cocok	GitFlow	GitHub Flow	Trunk-based

2.5 Protected Branches

Branch protection rules memastikan kode tidak bisa di-push langsung ke branch penting (main, develop).

Setup di GitHub

1. Repository → **Settings** → **Branches** → **Add branch protection rule**
2. Branch name pattern: `main`
3. Checklist:

```
☑ Require a pull request before merging
   ☑ Require approvals (1-2)
  ☑ Dismiss stale pull request approvals when new commits are pushed
   ☑ Require review from Code Owners

☑ Require status checks to pass before merging
   ☑ Require branches to be up to date

☑ Require conversation resolution first

☑ Include administrators (opsional)

☑ Restrict who can push to matching branches
```

Status Checks

Hubungkan GitHub Actions sebagai required check:

1. Di `.github/workflows/ci.yml`, beri nama job
2. Di branch protection, cari nama job → centang
3. PR tidak bisa merge kalau CI failed

Required Reviewers

- Minimal 1-2 orang approve sebelum merge
- Review dari code owner otomatis jika ada CODEOWNERS
- Stale review di-dismiss jika ada push baru

2.6 Code Owners

File `CODEOWNERS` di `.github/CODEOWNERS` — siapa yang auto-assign review berdasarkan path file.

Format

```
# Default owners untuk semua file
* @tim-lead @senior-dev

# Owner spesifik folder
/src/api/ @backend-team
/src/components/ @frontend-team
/src/database/ @dba-team

# Owner spesifik file
```

```
README.md @tech-writer
package.json @devops-team

# Multiple owners — semua harus review
/docs/ @tech-writer @frontend-team
```

Cara Setup

```
mkdir -p .github
cat > .github/CODEOWNERS << 'EOF'
* @lead-developer
/src/api/ @backend-engineer
/src/components/ @frontend-engineer
/docs/ @tech-writer
EOF

git add .github/CODEOWNERS
git commit -m "docs: add CODEOWNERS for team"
```

Rules

- File `.github/CODEOWNERS` harus di `main` branch
- Pattern evaluasi dari yang paling spesifik
- Username atau email GitHub (bisa team @org/team-name)
- Wildcard * untuk default
- Path relatif dari root repo

2.7 Diagram PR Workflow

```
sequenceDiagram
    participant D as Developer
    participant R as Repository
    participant C as CI/CD
    participant V as Reviewer

    D->>R: Push feature branch
    D->>R: Create Pull Request
    R->>C: Trigger CI workflow
    C->>R: Status: pending / fail / pass
    R->>V: Assign reviewer
    V->>R: Review code
    alt Changes requested
        V->>D: Request changes
```

```
        D->>R: Push new commits
        R->>V: Re-review
    else Approved
        V->>R: Approve PR
        D->>R: Merge PR
        R->>C: Deploy
    end
```

2.8 Latihan

Latihan 1: Setup Branch Protection

1. Buka repo latihan dari sesi 1
2. Settings → Branches → Add rule
3. Pattern: `main`
4. Checklist:
   - Require PR before merging
   - Require 1 approval
   - Dismiss stale reviews
   - Require conversation resolution
5. Save

Latihan 2: Create Pull Request

Part A: Feature Branch + PR

```bash
# 1. Pastikan branch main up-to-date
git checkout main
git pull origin main

# 2. Buat branch fitur
git checkout -b feature/profile-page

# 3. Buat file profile.html
cat > profile.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Profile Page</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: system-ui; display: flex; justify-content: center; align
```

```
    .card { background: white; border-radius: 16px; padding: 2rem; box-shadow: 0
    .avatar { width: 96px; height: 96px; border-radius: 50%; background: #2563eb
    h2 { color: #1e293b; }
    p { color: #64748b; margin-top: 0.5rem; }
  </style>
</head>
<body>
  <div class="card">
    <div class="avatar"></div>
    <h2>Nama Kamu</h2>
    <p>Frontend Developer</p>
    <p>✉ email@example.com</p>
  </div>
</body>
</html>
EOF

# 4. Commit dan push
git add profile.html
git commit -m "feat(profile): add profile page card component"
git push origin feature/profile-page
```

5. Buka GitHub → lihat banner “Compare & Pull Request” → klik
6. Isi PR description dengan template:
   - **What**: Tambah profile page dengan card component
   - **Why**: Fitur user profile untuk halaman dashboard
   - **How**: HTML + CSS inline, card component reusable
   - **Testing**: Buka `profile.html` di browser
   - **Screenshots**: (screenshot jika bisa)
7. Assign reviewer ke teman (atau diri sendiri untuk latihan)
8. Submit PR

Part B: Review PR

1. Buka tab **Files changed** di PR
2. Tinggalkan komentar di baris tertentu:
   - +1 komentar pujian
   - +1 komentar suggestion (misal: pindahkan CSS ke file terpisah)
   - +1 komentar question (misal: “Apa perlu responsive?”)
3. Klik **Finish review** → pilih **Comment** (belum approve)
4. Balas komentar sebagai author
5. Buat perubahan berdasarkan feedback:

```
# Pindahkan CSS ke file terpisah
git mv profile.html styles.css
# ... atau edit langsung
```

6. Push perubahan → komentar otomatis resolved

Part C: Merge

1. Pastikan semua conversation resolved
2. Dapatkan approval (atau approve sendiri untuk latihan)
3. Pilih merge strategy:
   - Coba **Squash and merge** → lihat commit message
   - Lihat hasil di graf: `git log --oneline --graph`
4. Hapus branch setelah merge (klik "Delete branch" di GitHub)

Latihan 3: Setup CODEOWNERS

```
# Di repo latihan
cat > .github/CODEOWNERS << 'EOF'
* @usernamekamu
/src/ @usernamekamu
/docs/ @usernamekamu
EOF

git add .github/CODEOWNERS
git commit -m "docs: add CODEOWNERS"
git push origin main
```

Checklist Output Sesi 2

- ☐ Branch protection rule untuk `main` sudah aktif
- ☐ Minimal 1 PR dibuat dengan deskripsi lengkap (what, why, how, testing)
- ☐ PR sudah di-review dengan minimal 3 komentar (praise, suggestion, question)
- ☐ Semua conversation di-resolved
- ☐ PR sudah di-merge (squash & merge atau merge commit)
- ☐ Branch fitur sudah dihapus setelah merge
- ☐ File `.github/CODEOWNERS` sudah ada

Referensi

- GitHub Pull Request Docs
- Best Practices for Code Review
- Conventional Comments
- Branch Protection Rules
- About CODEOWNERS

Sesi 3: Conflict Resolution

> Tangani merge conflict dengan percaya diri — resolve di IDE, rebase workflow, cherry-pick, revert, dan reset.

Durasi: 4 jam | **Output**: Conflict resolved PR + rebase practice

---

3.1 Merge Conflict Types

Conflict terjadi ketika Git tidak bisa menggabungkan perubahan secara otomatis.

Content Conflict (Paling Sering)

Dua branch mengubah baris yang sama di file yang sama.

```
<<<<<<< HEAD (main)
<h1>Selamat Datang</h1>
=======
<h1>Welcome</h1>
>>>>>>> feature/english
```

Penyebab: Dua developer edit baris yang sama.

Structural Conflict

File direstruktur — dipindah, di-rename, atau dihapus di satu branch, tapi diedit di branch lain.

```
CONFLICT (rename/delete): style.css deleted in main and renamed to src/styles.css i
```

Penyebab: Satu branch hapus file, branch lain rename.

Rename Conflict

File di-rename dengan nama berbeda di dua branch.

```
CONFLICT (rename/rename): about.html renamed to about-
us.html in branch A and team.html in branch B.
```

Penyebab: Dua branch rename file yang sama ke nama berbeda.

Binary Conflict

File binary (gambar, PDF, zip) diubah di dua branch.

```
CONFLICT (content): Merge conflict in logo.png
```

Penyebab: Dua branch mengedit binary file yang sama.

Tips Mencegah Conflict

- **PR kecil & fokus** — satu fitur per branch
- **Rebase sering** — jaga branch tetap up-to-date dengan `main`
- **Komunikasi tim** — kasih tahu kalau sedang edit file yang sama
- **Pull sebelum push** — selalu `git pull --rebase` sebelum push

3.2 Resolve Conflict di VS Code

VS Code punya **Merge Editor** bawaan — GUI terbaik untuk resolve conflict.

Cara Pakai Merge Editor

1. Ketika conflict terjadi:

   ```
   git merge feature/login
   # Auto-merging index.html
   # CONFLICT (content): Merge conflict in index.html
   ```

2. Buka file di VS Code — lihat UI conflict markers
3. Klik tombol **“Resolve in Merge Editor”** (di kanan bawah editor)
4. Merge Editor menampilkan 3 panel:
   - **Kiri**: Isi dari branch kita (Incoming)
   - **Kanan**: Isi dari branch lain (Current)
   - **Bawah**: Hasil merge (Result)
5. Pilih aksi per konflik:
   - **Accept Current** — pakai punya branch kita
   - **Accept Incoming** — pakai punya branch lawan
   - **Accept Both** — gabungkan keduanya
   - Manual edit di panel Result
6. Klik **“Complete Merge”** setelah selesai
7. Stage + commit:

```
git add index.html
git commit -m "chore: resolve merge conflict in index.html"
```

Conflict Markers Manual

Jika tidak pakai VS Code, edit manual:

```
<<<<<<< HEAD (Current — branch yang kamu tuju)
<h1>Selamat Datang</h1>
======= (Pemisah)
<h1>Welcome</h1>
>>>>>>> feature/english (Incoming — branch yang di-merge)
```

Pilih satu versi atau gabung:

```
<h1>Selamat Datang | Welcome</h1>
```

Hapus semua marker (<<<<<<<, =======, >>>>>>>) setelah edit.

Diff Viewer di Terminal

```
# Lihat perbedaan antar branch
git diff main..feature/login -- index.html

# Lihat conflict dalam konteks
git diff

# Setelah resolve, verify
git diff --check
```

3.3 Rebase Workflow

Rebase menulis ulang histori branch — memindahkan base branch ke titik yang lebih baru.

Kenapa Rebase?

```
Sebelum rebase:
      A---B---C feature/login
     /
D---E---F---G main

Sesudah rebase:
              A'---B'---C' feature/login
             /
D---E---F---G main
```

Keuntungan: Histori linear, tidak ada merge commit.

Kerugian: Force push diperlukan, hati-hati di branch bersama.

Interactive Rebase

```
# Rebase 3 commit terakhir
git rebase -i HEAD~3

# Atau rebase ke main
git rebase -i main
```

Editor terbuka dengan opsi per commit:

```
pick a1b2c3 feat: add login form
pick d4e5f6 feat: add validation
pick g7h8i9 fix: validation error
```

Rebase Commands

Command	Fungsi
`pick`	Pakai commit apa adanya
`reword`	Ubah pesan commit
`edit`	Hentikan rebase untuk amend
`squash`	Gabung dengan commit sebelumnya
`fixup`	Gabung, buang pesan commit
`drop`	Hapus commit
`exec`	Jalankan perintah shell

Contoh: Squash 3 Commit Jadi 1

```
pick a1b2c3 feat: add login form
squash d4e5f6 feat: add validation
squash g7h8i9 fix: validation error
```

Hasil: Satu commit dengan pesan `feat: add login form`.

Contoh: Fixup (buang pesan)

```
pick a1b2c3 feat: add login form
fixup d4e5f6 wip: fix validation
fixup g7h8i9 oops forgot this
```

Hasil: Semua perubahan di commit `a1b2c3`, pesan `d4e5f6` dan `g7h8i9` dibuang.

Rebase Workflow Harian

```
# 1. Saat mulai kerja — rebase branch fitur ke main terbaru
git checkout main
```

```
git pull origin main
git checkout feature/login
git rebase main

# 2. Kalau ada conflict — resolve satu per satu
# Git akan stop di setiap commit yang conflict
git status           # lihat file conflict
# edit + resolve
git add <file>
git rebase --continue

# 3. Kalau mau skip commit yang conflict
git rebase --skip

# 4. Kalau mau batalin rebase
git rebase --abort

# 5. Setelah rebase selesai — force push
git push origin feature/login --force-with-lease
```

Penting: `--force-with-lease`

Selalu pakai `--force-with-lease` bukan `--force`:

```
# Aman — cek apakah ada perubahan remote yang tidak diketahui
git push --force-with-lease

# Berbahaya — timpa apapun di remote
git push --force  #  Hindari
```

3.4 Cherry-pick, Revert, Reset

Cherry-pick

Ambil commit spesifik dari branch lain ke branch sekarang.

```
# Lihat commit di branch lain
git log feature/login --oneline
# a1b2c3 feat: add login form
# d4e5f6 feat: add validation

# Cherry-pick satu commit ke branch sekarang
git checkout main
git cherry-pick a1b2c3
```

```
# Cherry-pick range commit
git cherry-pick a1b2c3..d4e5f6

# Dengan opsi
git cherry-pick -x a1b2c3         # tambah "(cherry picked from commit ...)"
git cherry-pick --no-commit a1b2c3 # stage aja, jangan auto-commit
```

Use case: Ambil hotfix dari branch release ke main tanpa merge seluruh branch.

Revert

Buat commit baru yang membalikkan perubahan commit sebelumnya. **Aman untuk histori bersama.**

```
# Revert satu commit
git revert a1b2c3
# Membuat commit baru "Revert 'feat: add login form'"

# Revert tanpa auto-commit
git revert --no-commit a1b2c3

# Revert range
git revert HEAD~3..HEAD

# Revert merge commit (perlu -m untuk parent)
git revert -m 1 <merge-commit-hash>
```

Use case: Rollback fitur di produksi tanpa merusak histori.

Reset

Pindahkan pointer branch ke commit tertentu. **Tiga mode:**

Mode	Working Directory	Staging Area	Commit History
`--soft`	□ Tidak berubah	□ Tidak berubah	□ Pindah
`--mixed` (default)	□ Tidak berubah	□ Unstage	□ Pindah
`--hard`	□ Hapus semua	□ Hapus semua	□ Pindah

```
# Soft — ubah commit terakhir, files tetap staged
git reset --soft HEAD~1

# Mixed — ubah commit terakhir, files unstaged
git reset HEAD~1

# Hard — buang semua perubahan, balik ke commit sebelumnya
```

```
git reset --hard HEAD~1

# Hati-hati! Kalau sudah push, gunakan revert, bukan reset
```

Use case: - `--soft`: “Ups, lupa masukin file di commit terakhir” - `--mixed`: “Ups, commit salah file” - `--hard`: “Reset lokal ke kondisi bersih”

Tabel Perbandingan

Operasi	Histori	Aman untuk publik?	Force push?
`cherry-pick`	Tambah commit baru	□ Ya	□ Tidak
`revert`	Tambah commit baru	□ Ya	□ Tidak
`reset --soft`	Tulis ulang	□ Hanya lokal	□ Perlu
`reset --hard`	Tulis ulang	□ Hanya lokal	□ Perlu

> **Aturan emas**: Jangan `reset` commit yang sudah di-push ke branch bersama. Pakai `revert`.

3.5 Tips Tim

Komunikasi

- **Sebelum rebase force push**: kasih tahu tim “Saya rebase branch X, jangan push dulu”
- **Conflict besar**: panggil author branch lain untuk diskusi langsung
- **Gunakan draft PR**: untuk branch yang masih WIP, jangan minta review dulu

Small PRs

- Maksimal 200-300 lines per PR
- Satu PR = satu concern
- PR kecil = conflict lebih jarang, review lebih cepat

Rebase Often

```
# Minimal sehari sekali
git checkout main
git pull origin main
git checkout feature/login
git rebase main
```

Git Aliases untuk Produktivitas

```
# Tambah ke ~/.gitconfig atau ~/.bashrc
git config --global alias.lg "log --graph --oneline --all"
git config --global alias.undo "reset --soft HEAD~1"
git config --global alias.amend "commit --amend --no-edit"
git config --global alias.please "push --force-with-lease"
```

3.6 Latihan

Latihan 1: Create Merge Conflict

Setup

```
# 1. Clone atau masuk ke repo latihan
cd ~/Projects/latihan-git-workflow  # atau repo manapun

# 2. Buat file bersama
cat > team-list.txt << 'EOF'
Anggota Tim:
- Alice
- Bob
EOF

git add team-list.txt
git commit -m "feat: add team list with Alice and Bob"

git push origin main
```

Part A: Conflict Content

```
# 3. Dua branch edit baris yang sama
git checkout main
git checkout -b feature/tim-mark
# Edit team-list.txt — tambah Mark setelah Bob
cat > team-list.txt << 'EOF'
Anggota Tim:
- Alice
- Bob
- Mark
EOF

git add team-list.txt
git commit -m "feat(team): add Mark to team list"
git push origin feature/tim-mark
```

```
# 4. Branch lain
git checkout main
git checkout -b feature/tim-julia
# Edit team-list.txt — tambah Julia setelah Bob
cat > team-list.txt << 'EOF'
Anggota Tim:
- Alice
- Bob
- Julia
EOF

git add team-list.txt
git commit -m "feat(team): add Julia to team list"
git push origin feature/tim-julia
```

Part B: Resolve Conflict

```
# 5. Merge feature/tim-mark dulu ke main
git checkout main
git merge feature/tim-mark  # sukses
git push origin main

# 6. Coba merge feature/tim-julia → CONFLICT!
git merge feature/tim-julia
# CONFLICT (content): Merge conflict in team-list.txt
```

7. Buka VS Code → resolve conflict:
    - Accept both changes
    - Hasil: Alice, Bob, Mark, Julia
    - Simpan

```
# 8. Stage + commit
git add team-list.txt
git commit -m "chore: resolve merge conflict team list"
git push origin main
```

Latihan 2: Rebase Feature Branch

```
# 1. Buat branch fitur baru
git checkout main
git checkout -b feature/about-page

# 2. Buat file
cat > about.html << 'EOF'
<!DOCTYPE html>
```

```
<html><body><h1>Tentang Kami</h1></body></html>
EOF

git add about.html
git commit -m "feat(about): add about page skeleton"

# 3. Sementara itu, main maju (simulasi tim lain push)
git checkout main
cat > README.md << 'EOF'
# Latihan Git Workflow
Selamat datang di repo latihan!
EOF
git add README.md
git commit -m "docs: update README with intro"
git push origin main

# 4. Rebase branch fitur ke main terbaru
git checkout feature/about-page
git rebase main
# Jika conflict, resolve + git rebase --continue

# 5. Force push
git push origin feature/about-page --force-with-lease

# 6. Buat PR, merge, selesai
```

Latihan 3: Interactive Rebase — Squash & Fixup

```
# 1. Buat branch
git checkout main
git checkout -b feature/squash-practice

# 2. Buat 3 commit
echo "# Project" > project.md
git add project.md
git commit -m "feat: init project"

echo "## Installation" >> project.md
git add project.md
git commit -m "wip: add install section"

echo "## Usage" >> project.md
git add project.md
git commit -m "wip: add usage section"
```

```
# 3. Squash jadi 1 commit
git rebase -i HEAD~3
# Ubah: pick → squash untuk commit 2 dan 3
# Simpan → edit pesan jadi "docs: add project README with install and usage"

# 4. Verifikasi
git log --oneline -3

# 5. Force push
git push origin feature/squash-practice --force-with-lease
```

Latihan 4: Cherry-pick & Revert

```
# Cherry-pick
git checkout main
git log feature/about-page --oneline
git cherry-pick <hash-commit-about-page>
# Commit about page masuk ke main tanpa merge seluruh branch

# Revert
git log --oneline -5
git revert HEAD  # revert commit terakhir
# Commit revert dibuat otomatis
git log --oneline -5  # lihat commit revert baru
```

Checklist Output Sesi 3

- ☐ Berhasil membuat dan resolve merge conflict (content conflict)
- ☐ Berhasil rebase feature branch ke main terbaru
- ☐ Berhasil interactive rebase: squash 3 commit jadi 1
- ☐ Berhasil cherry-pick commit dari branch lain
- ☐ Berhasil revert commit
- ☐ Paham perbedaan reset –soft, –mixed, –hard
- ☐ Paham kapan pakai `--force-with-lease`

Referensi

- Git Branching - Basic Branching and Merging
- VS Code Merge Editor
- Git Tools - Rewriting History
- Git Tools - Reset Demystified
- Oh shit, git! — penyelamat darurat git

Sesi 4: GitHub Actions & Project Management

> Otomatiskan CI/CD pipeline, setup GitHub Projects board, dan kelola issues & PR dengan template profesional.

Durasi: 4 jam | **Output**: CI workflow running + GitHub Project board

---

4.1 GitHub Actions Overview

GitHub Actions = CI/CD bawaan GitHub. Jalankan workflow otomatis berdasarkan event di repository.

Konsep Dasar

Istilah	Penjelasan
Workflow	Satu file YAML di `.github/workflows/` — definisi pipeline
Job	Kumpulan step yang jalan di runner yang sama
Step	Satu perintah (run action atau shell command)
Action	Unit reusable — bisa dari GitHub Marketplace atau buat sendiri
Runner	Server yang menjalankan workflow (GitHub-hosted atau self-hosted)
Event	Trigger — push, pull_request, schedule, dll
Matrix	Jalanin job yang sama dengan multiple versi

Struktur Workflow

```yaml
name: CI Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
```

```yaml
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: npm ci
      - run: npm test
```

4.2 Workflow YAML — Komponen Detail

Event Triggers

```yaml
on:
  # Push ke branch tertentu
  push:
    branches: [main, develop]
    paths:
      - 'src/**'
      - '!docs/**'

  # Pull request ke branch tertentu
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]

  # Schedule (cron) — format UTC
  schedule:
    - cron: '0 6 * * 1'  # Setiap Senin jam 6 pagi UTC

  # Manual trigger dari UI
  workflow_dispatch:
    inputs:
      environment:
        description: 'Deploy environment'
        required: true
        default: 'staging'
        type: choice
        options:
          - staging
          - production
```

Jobs & Dependencies

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Linting..."

  test:
    needs: lint  # Tunggu lint selesai
    runs-on: ubuntu-latest
    steps:
      - run: echo "Testing..."

  build:
    needs: [lint, test]  # Tunggu keduanya
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building..."
```

Matrix Strategy

Jalankan job yang sama dengan multiple versi / konfigurasi:

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [16, 18, 20]
        os: [ubuntu-latest, windows-latest]
        include:
          - node-version: 18
            os: ubuntu-latest
            coverage: true
        exclude:
          - node-version: 16
            os: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
      - run: npm ci
      - run: npm test
      - if: matrix.coverage
```

```
        run: npm run coverage
```

Environment Variables & Secrets

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      NODE_ENV: production
    steps:
      - name: Deploy
        run: |
          echo "Deploying to ${{ env.NODE_ENV }}"
        env:
          APP_SECRET: ${{ secrets.APP_SECRET }}
```

Set **Repository Secrets** di: Settings → Secrets and variables → Actions

---

4.3 CI Pipeline (Lint → Type Check → Test → Build → Coverage)

Pipeline CI lengkap untuk Node.js + TypeScript:

File `.github/workflows/ci.yml`:

```
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    name: Lint & Format
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
```

```yaml
      with:
        node-version: 20
        cache: 'npm'
    - run: npm ci
    - run: npm run lint
    - run: npm run format:check

type-check:
  name: TypeScript Type Check
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: 'npm'
    - run: npm ci
    - run: npm run type-check

test:
  name: Unit & Integration Tests
  needs: [lint, type-check]
  runs-on: ubuntu-latest
  strategy:
    matrix:
      node-version: [18, 20]
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}
        cache: 'npm'
    - run: npm ci
    - run: npm test
    - name: Upload Coverage
      uses: actions/upload-artifact@v4
      with:
        name: coverage-${{ matrix.node-version }}
        path: coverage/

build:
  name: Build
  needs: [test]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
```

```yaml
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run build
      - name: Upload Build Artifact
        uses: actions/upload-artifact@v4
        with:
          name: build
          path: dist/

  coverage-report:
    name: Coverage Report
    needs: [test]
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run coverage
      - uses: davelosert/vitest-coverage-report-action@v2
        with:
          json-summary-path: coverage/coverage-summary.json
```

Pipeline Visual

```
graph LR
    A[Push / PR] --> B[Lint & Format]
    A --> C[Type Check]
    B --> D[Test Node 18]
    B --> E[Test Node 20]
    C --> D
    C --> E
    D --> F[Build]
    E --> F
    F --> G[Coverage Report]
    G --> H[ Ready to Merge]
```

4.4 CD Pipeline (Build → Push → Deploy)

Deploy ke Vercel

File `.github/workflows/deploy-vercel.yml`:

```yaml
name: Deploy to Vercel

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run build
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
          vercel-args: '--prod'
```

Deploy ke Railway

File `.github/workflows/deploy-railway.yml`:

```yaml
name: Deploy to Railway

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Railway CLI
```

```
        run: npm i -g @railway/cli
      - name: Deploy
        run: railway up --service ${{ secrets.RAILWAY_SERVICE }}
        env:
          RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}
```

Deploy ke GitHub Pages

File `.github/workflows/deploy-pages.yml`:

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist/

  deploy:
    needs: build
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

4.5 GitHub Projects

GitHub Projects = Kanban board untuk manage task tim.

Setup Project Board

1. Repository → **Projects** → **Create project**
2. Pilih template: **Feature planning** atau **Bug triage** atau **Blank**
3. Beri nama: "Development Sprint 1"

Columns Standar

Column	Isi
Backlog	Ide/task yang belum diprioritaskan
To Do	Task siap dikerjakan
In Progress	Sedang dikerjakan
In Review	PR sudah dibuat, menunggu review
Done	Selesai dan sudah di-merge

Labels

Buat label di repo → Issues → Labels:

Label	Warna	Deskripsi
`bug`	`#d73a4a`	Something isn't working
`enhancement`	`#a2eeef`	New feature or request
`documentation`	`#0075ca`	Improvements or additions to documentation
`good first issue`	`#7057ff`	Good for newcomers
`help wanted`	`#008672`	Extra attention is needed
`priority: high`	`#b60205`	Critical priority
`priority: medium`	`#fbca04`	Medium priority
`priority: low`	`#0e8a16`	Low priority

Milestones

1. Repository → **Issues** → **Milestones** → **Create a milestone**
2. Isi title, due date, description
3. Assign issues ke milestone

Automation Rules

GitHub Projects bisa otomatis move cards:

Trigger	Aksi
Issue opened →	Move ke **To Do**
PR opened →	Move ke **In Review**
PR merged →	Move ke **Done**
Issue/PR closed →	Move ke **Done**

Setup: Di Project board → ⚙ Settings → **Automation** → Add workflow

---

4.6 Issue & PR Templates

Issue Template: Bug Report

File `.github/ISSUE_TEMPLATE/bug_report.md`:

```
---
name: Bug report
about: Create a report to help us improve
title: '[BUG] '
labels: bug
assignees: ''
---

## Describe the Bug
<!-- Clear description of the bug -->

## To Reproduce
Steps:
1. Go to '...'
2. Click on '...'
3. See error

## Expected Behavior
<!-- What should happen -->

## Screenshots
<!-- If applicable -->

## Environment
- OS: [e.g. Windows, macOS]
```

```
- Browser: [e.g. Chrome, Firefox]
- Version: [e.g. 1.0.0]

## Additional Context
<!-- Add any other context -->
```

Issue Template: Feature Request

File .github/ISSUE_TEMPLATE/feature_request.md:

```
---
name: Feature request
about: Suggest an idea for this project
title: '[FEATURE] '
labels: enhancement
assignees: ''
---

## Problem
<!-- Is your feature request related to a problem? Describe -->

## Solution
<!-- Describe the solution you'd like -->

## Alternatives
<!-- Describe alternatives you've considered -->

## Additional Context
<!-- Add any other context or screenshots -->
```

Setup Semua Template

```
# Buat struktur folder
mkdir -p .github/ISSUE_TEMPLATE

# Download template dari repo ini atau buat manual
# ...

# Commit
git add .github/
git commit -m "docs: add issue and PR templates"
git push origin main
```

4.7 Diagram: CI/CD Pipeline

```
graph TD
    A[Push to main / PR] --> B{CI Pipeline}
    B --> C[Lint & Format]
    B --> D[Type Check]
    C & D --> E[Unit Tests]
    E --> F{Build}
    F --> G{CD Pipeline}

    G --> H[Deploy Staging]
    G --> I[Deploy Production]

    H --> J[Staging URL]
    I --> K[Production URL]

    style A fill:#2563eb,color:#fff
    style B fill:#f59e0b,color:#fff
    style G fill:#10b981,color:#fff
    style K fill:#10b981,color:#fff
```

4.8 Latihan

Latihan 1: Setup CI Workflow

Setup Project Sederhana

```bash
# Buat project Node.js minimal
cd ~/Projects/latihan-git-workflow  # atau folder baru

# Init project (kalau belum ada package.json)
npm init -y

# Install tools
npm install --save-dev vitest @biomejs/biome

# Setup test file
mkdir src
cat > src/sum.js << 'EOF'
export function sum(a, b) {
  return a + b;
}
EOF

cat > src/sum.test.js << 'EOF'
```

```
import { describe, it, expect } from 'vitest';
import { sum } from './sum';

describe('sum', () => {
  it('adds 1 + 2 = 3', () => {
    expect(sum(1, 2)).toBe(3);
  });

  it('adds negative numbers', () => {
    expect(sum(-1, -1)).toBe(-2);
  });
});
EOF

# Update package.json scripts
# "test": "vitest run"
# "type": "module"
```

File package.json (update):

```
{
  "name": "latihan-git-workflow",
  "type": "module",
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest",
    "lint": "biome check src/",
    "format": "biome format --write src/"
  },
  "devDependencies": {
    "@biomejs/biome": "^1.8.0",
    "vitest": "^1.6.0"
  }
}
```

Buat CI Workflow

```
mkdir -p .github/workflows
```

File .github/workflows/ci.yml:

```
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18, 20]

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

```bash
# Commit dan push
git add .
git commit -m "ci: add CI workflow with lint and test"
git push origin main
```

3. Buka GitHub → **Actions** tab → lihat workflow running
4. Klik workflow → lihat job test dengan matrix [18, 20]
5. Verifikasi semua green

Tambah Badge CI ke README

```
![CI Pipeline](https://github.com/username/latihan-git-workflow/actions/workflow
```

Latihan 2: Setup GitHub Project Board

1. Buka repo → **Projects** → **Create project** → **Blank**
2. Nama: "Development Sprint"
3. Buat columns: **Backlog**, **To Do**, **In Progress**, **In Review**, **Done**
4. Buat labels (Settings → Labels → New label):
   - bug (red #d73a4a)
   - enhancement (green #a2eeef)
   - `good first issue` (purple #7057ff)

Latihan 3: Create Issues & Milestones

1. Buka **Issues** → **Milestones** → **New milestone**
   - Title: "Sprint 1 - Git Workflow"
   - Due date: +2 minggu

- Description: “Implementasi branching strategy + CI/CD”

2. Buat Issues:

Issue 1: “Setup branching strategy documentation” - Labels: documentation - Project: Development Sprint → To Do - Milestone: Sprint 1

Issue 2: “Add unit tests for utility functions” - Labels: enhancement, good first issue - Project: Development Sprint → To Do - Assignees: diri sendiri

Issue 3: “Fix broken navigation on mobile” - Labels: bug - Priority: priority: high - Project: Development Sprint → To Do

Latihan 4: Setup Issue & PR Templates

```
# Buat template structure
mkdir -p .github/ISSUE_TEMPLATE

# Download template files (copy dari sesi 4.6)
# Atau buat manual
```

File `.github/ISSUE_TEMPLATE/bug_report.md` dan `feature_request.md` (copy dari konten 4.6).

File `.github/pull_request_template.md` (copy dari sesi 2.2).

```
git add .github/
git commit -m "docs: add issue and PR templates"
git push origin main
```

Verify: Buka repo → **Issues** → **New issue** — lihat template muncul.

Latihan 5: Uji CI Pipeline

1. Buat branch baru:

```
git checkout -b feature/test-ci
```

2. Edit file, buat test fail:

```
cat > src/sum.test.js << 'EOF'
import { describe, it, expect } from 'vitest';
import { sum } from './sum';

describe('sum', () => {
  it('should fail', () => {
    expect(sum(1, 2)).toBe(999); // FAIL!
  });
```

```
});
EOF
```

3. Commit + push → buat PR:

```
git add src/sum.test.js
git commit -m "test: intentionally failing test"
git push origin feature/test-ci
```

4. Buka GitHub → Create PR
5. Lihat **CI Pipeline** — akan FAIL
6. PR tidak bisa merge (karena branch protection + status check fails)
7. Fix test, push ulang → CI PASS → merge

Checklist Output Sesi 4

- ☐ CI workflow (lint + test + build) berjalan otomatis di GitHub Actions
- ☐ Badge CI status terlihat di README
- ☐ GitHub Project board dengan columns dan cards
- ☐ Minimal 3 issues dengan labels dan milestone
- ☐ File `.github/ISSUE_TEMPLATE/bug_report.md` dan `feature_request.md`
- ☐ File `.github/pull_request_template.md`
- ☐ PR gagal merge karena CI failed — dan berhasil setelah fix
- ☐ Paham perbedaan event trigger (push vs pull_request vs schedule)

Referensi

- GitHub Actions Documentation
- Workflow Syntax
- GitHub Actions Marketplace
- Vercel Action
- Railway CLI
- GitHub Projects Docs
- Creating Issue Templates